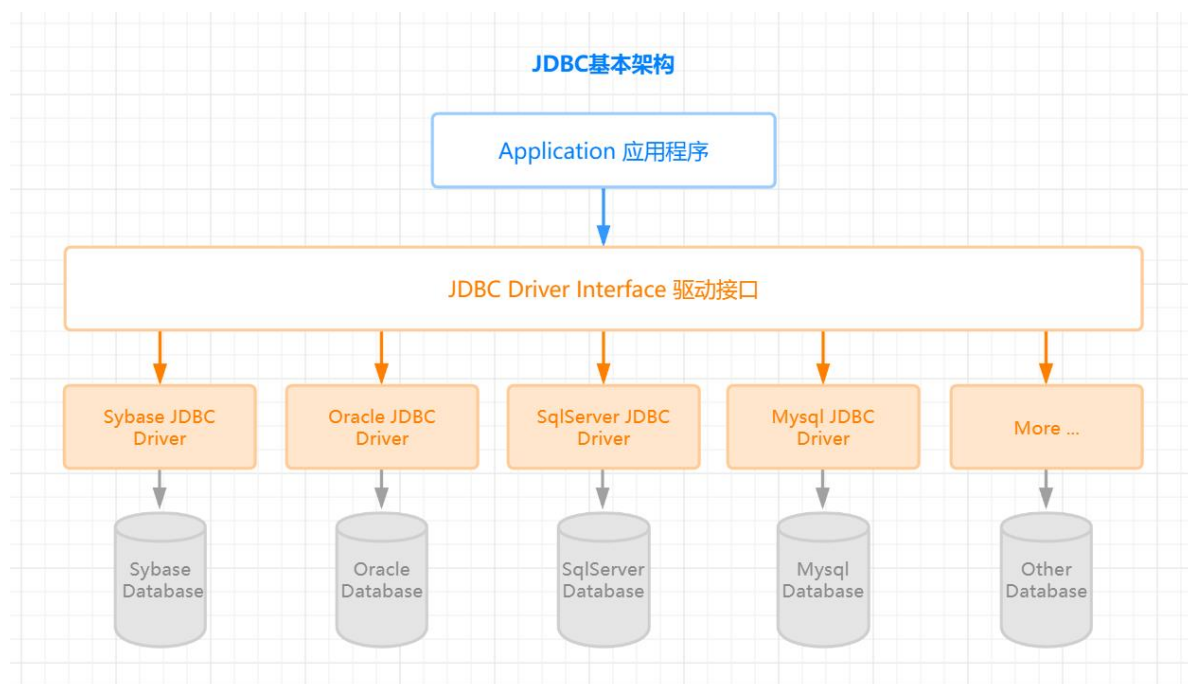


一、JDBC概述

1.1、JDBC介绍

JDBC (Java Data Base Connectivity,java数据库连接) 是一种用于执行SQL语句的Java API, 可以为多种关系数据库提供统一访问, 它由一组用Java语言编写的类和接口组成。JDBC提供了一种基准, 据此可以构建更高级的工具和接口, 使数据库开发人员能够编写数据库应用程序, 同时, JDBC也是个商标名。

JDBC流程如下:



1.2、JDBC优缺点

优点: 直接底层操作, 提供了很简单、便捷的访问数据库的方法, 跨平台性比较强。灵活性比较强, 可以写很复杂的SQL语句。

缺点: 因为JAVA是面向对象的, JDBC没有做到使数据能够面向对象的编程, 使程序员的思考仍停留在SQL语句上。

操作比较繁琐, 很多代码需要重复写很多次。如果遇到批量操作, 频繁与数据库进行交互, 容易造成效率的下降。

二、Mybatis概述

2.1、什么是Mybatis?

MyBatis 是一款优秀的持久层框架，它支持自定义 SQL、存储过程以及高级映射。MyBatis 免除了几乎所有的 JDBC 代码以及设置参数和获取结果集的工作。MyBatis 可以通过简单的 XML 或注解来配置和映射原始类型、接口和 Java POJO（Plain Old Java Objects，普通老式 Java 对象）为数据库中的记录。

mybatis中文官网:<https://mybatis.net.cn/index.html>

2.2、mybatis框架优缺点

优点:

- 1、与JDBC相比，减少了50%的代码量
- 2、最简单的持久化框架，简单易学
- 3、SQL代码从程序代码中彻底分离出来，可以重用
- 4、提供XML标签，支持编写动态SQL
- 5、提供映射标签，支持对象与数据库的ORM字段关系映射

缺点:

- 1、SQL语句编写工作量大，熟练度要高
- 2、数据库移植性比较差，如果需要切换数据库的话，SQL语句会有很大的差异

三、mybatis使用

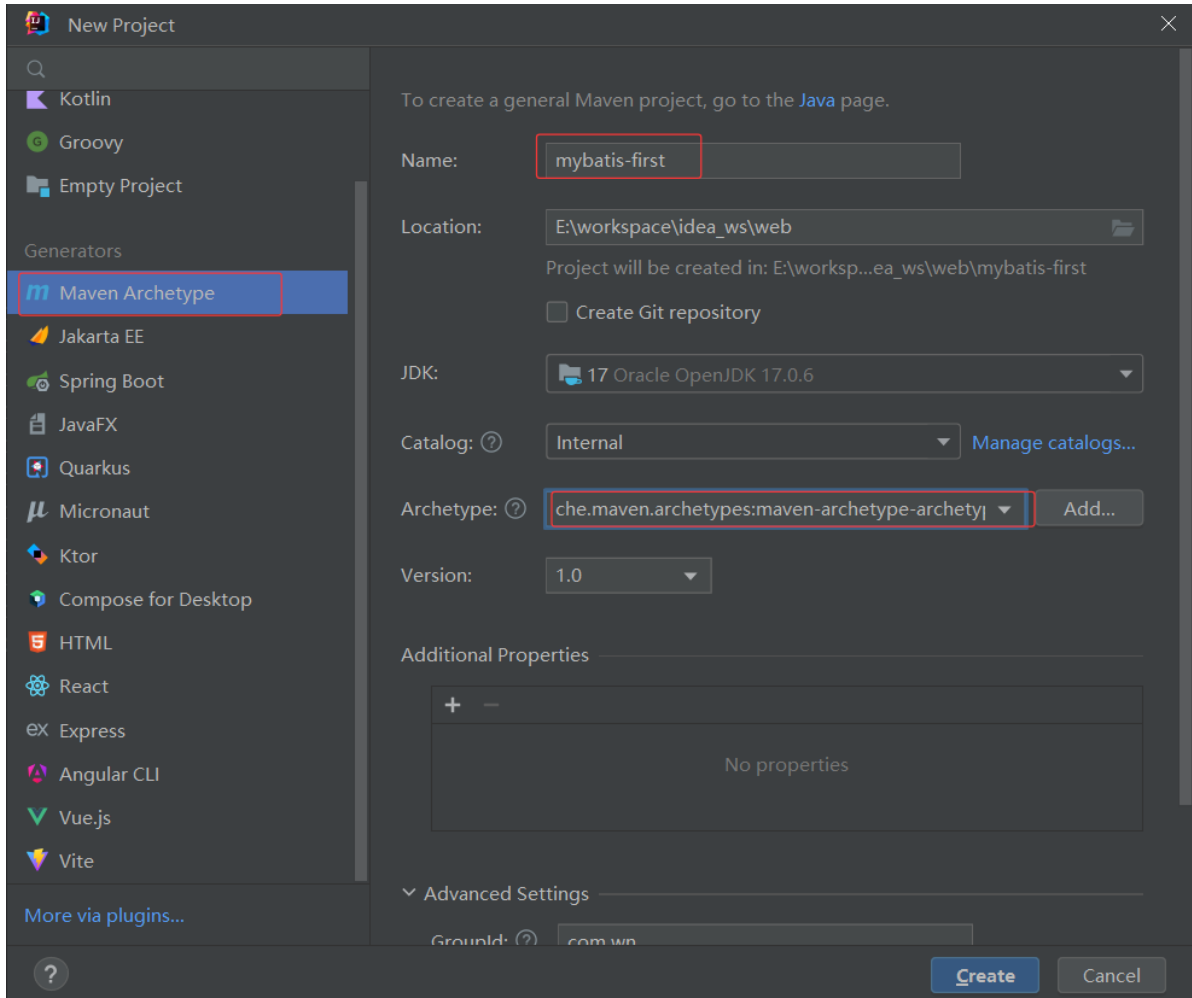
3.1、mybatis入门案例

3.1.1、创建数据表

```
1  -- -----
2  -- Table structure for `dept`
3  -- -----
4  DROP TABLE IF EXISTS `dept`;
5  CREATE TABLE `dept` (
6    `deptno` int NOT NULL,
7    `dname` varchar(15) DEFAULT NULL,
8    `loc` varchar(30) DEFAULT NULL,
9    PRIMARY KEY (`deptno`)
10 ) ENGINE=InnoDB DEFAULT CHARSET=utf8;
11
12 -- -----
13 -- Records of dept
14 -- -----
```

```
15 INSERT INTO `dept` VALUES ('10', '营销部', '上海');
16 INSERT INTO `dept` VALUES ('20', '财务部', '北京');
17 INSERT INTO `dept` VALUES ('30', '策划部', '杭州');
18 INSERT INTO `dept` VALUES ('40', '人事部', '成都');
19 INSERT INTO `dept` VALUES ('50', '保安部', '嘉兴');
20 INSERT INTO `dept` VALUES ('60', '公关部', '东莞');
```

3.1.2、Idea创建Java项目



3.1.3、pom.xml导入依赖

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/maven-v4_0_0.xsd">
3     <modelVersion>4.0.0</modelVersion>
4
5     <!-- 当前项目GAV坐标-->
6     <groupId>com.wn</groupId>
7     <artifactId>mybatis-first</artifactId>
8     <version>1.0-SNAPSHOT</version>
9     <name>Archetype - mybatis-first</name>
```

```
10 <url>http://maven.apache.org</url>
11
12 <properties>
13   <java.version>17</java.version>
14 </properties>
15 <!-- 添加项目依赖-->
16 <dependencies>
17   <!-- 引入mysql驱动依赖 -->
18   <dependency>
19     <groupId>mysql</groupId>
20     <artifactId>mysql-connector-java</artifactId>
21     <version>8.0.20</version>
22   </dependency>
23   <!-- 引入mybatis框架依赖 -->
24   <dependency>
25     <groupId>org.mybatis</groupId>
26     <artifactId>mybatis</artifactId>
27     <version>3.5.10</version>
28   </dependency>
29   <!-- Lombok依赖 -->
30   <dependency>
31     <groupId>org.projectlombok</groupId>
32     <artifactId>lombok</artifactId>
33     <version>1.18.30</version>
34   </dependency>
35 </dependencies>
36 <build>
37   <plugins>
38     <plugin>
39       <artifactId>maven-compiler-plugin</artifactId>
40       <version>3.1</version>
41       <configuration>
42         <!--当前jdk版本-->
43         <source>17</source>
44         <target>17</target>
45       </configuration>
46     </plugin>
47   </plugins>
48   <!-- maven运行时，默认不会加载src/main/java目录下的xml文件，通过以下配置方可解
决-->
49   <resources>
50     <resource>
51       <directory>src/main/java</directory>
52       <includes>
53         <include>**/*.properties</include>
54         <include>**/*.xml</include>
55       </includes>
56       <filtering>true</filtering>
57     </resource>
58     <resource>
59       <directory>src/main/resources</directory>
60       <includes>
61         <include>**/*.*</include>
62       </includes>
63       <filtering>true</filtering>
```

```

64     </resource>
65     </resources>
66 </build>
67 </project>

```

3.1.4、创建表对应实体类

Dept.java类:

```

1  /**
2   * 部门类
3   */
4  @Data
5  @NoArgsConstructor
6  @AllArgsConstructor
7  public class Dept {
8
9      /**部门编号*/
10     private Integer deptno;
11     /**部门名称*/
12     private String dname;
13     /**部门地址*/
14     private String loc;
15 }

```

3.1.5、db.properties配置文件

在src/main/resources目录下创建db.properties配置文件

```

1  jdbcDriver=com.mysql.cj.jdbc.Driver
2  jdbcUrl=jdbc:mysql://localhost:3306/mydb2?serverTimezone=GMT%2B8
3  jdbcUsername=root
4  jdbcPwd=123456

```

3.1.6、mybatis核心配置文件

在src/main/resources/下创建mybatis-config.xml

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE configuration
3      PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-config.dtd">
5  <configuration>
6      <environments default="development">
7          <environment id="development">
8              <transactionManager type="JDBC"/>
9              <dataSource type="POOLED">
10                 <property name="driver" value="com.mysql.cj.jdbc.Driver"/>

```

```

11         <property name="url"
value="jdbc:mysql://localhost:3306/mydb2?serverTimezone=GMT%2B8"/>
12         <property name="username" value="root"/>
13         <property name="password" value="123456"/>
14     </dataSource>
15 </environment>
16 </environments>
17 <mappers>
18     <mapper resource="org/wn/dao/DeptDao.xml"/>
19 </mappers>
20 </configuration>

```

3.1.7、创建sql映射文件

在org/wn/dao目录下创建DeptDao.xml文件，内容如下：

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <!--
6      namespace: 命令空间，本案例中此处可以任意填写。但是到实际开发只能是对应dao层类的全路
径
7  -->
8  <mapper namespace="dpet">
9      <!-- 根据id查询部门信息
10         select: 所有查询操作都使用此标签
11         id: 唯一标识符，此案例中可以任意取值，但是后期实际开发只能是对应接口中的 方法
名称
12         resultType: 指定查询到的数据最终封装的java类型
13         parameterType: 指定参数类型
14     -->
15     <select id="findById" resultType="org.wn.pojo.Dept" parameterType="int">
16         select * from dept where deptno=#{deptno}
17     </select>
18 </mapper>

```

3.1.8、创建测试类

```

1  /**
2   * 测试类
3   */
4  public class MybatisTest {
5      public static void main(String[] args) throws IOException {
6          //1.加载mybatis核心配置文件，创建一个SqlSessionFactory工厂类对象
7          InputStream inputStream = Resources.getResourceAsStream("mybatis-
config.xml");
8          SqlSessionFactory sqlSessionFactory = new
SqlSessionFactoryBuilder().build(inputStream);
9          //2.通过sqlSessionFactory创建sqlSession会话对象

```

```

10     SqlSession sqlSession = sqlSessionFactory.openSession();
11     //3.通过sqlSession发送sql语句,自动封装数据为指定类型并返回
12     Dept dept = sqlSession.selectOne("dpet.findById",30);
13     System.out.println(dept);
14 }
15 }

```

启动类:

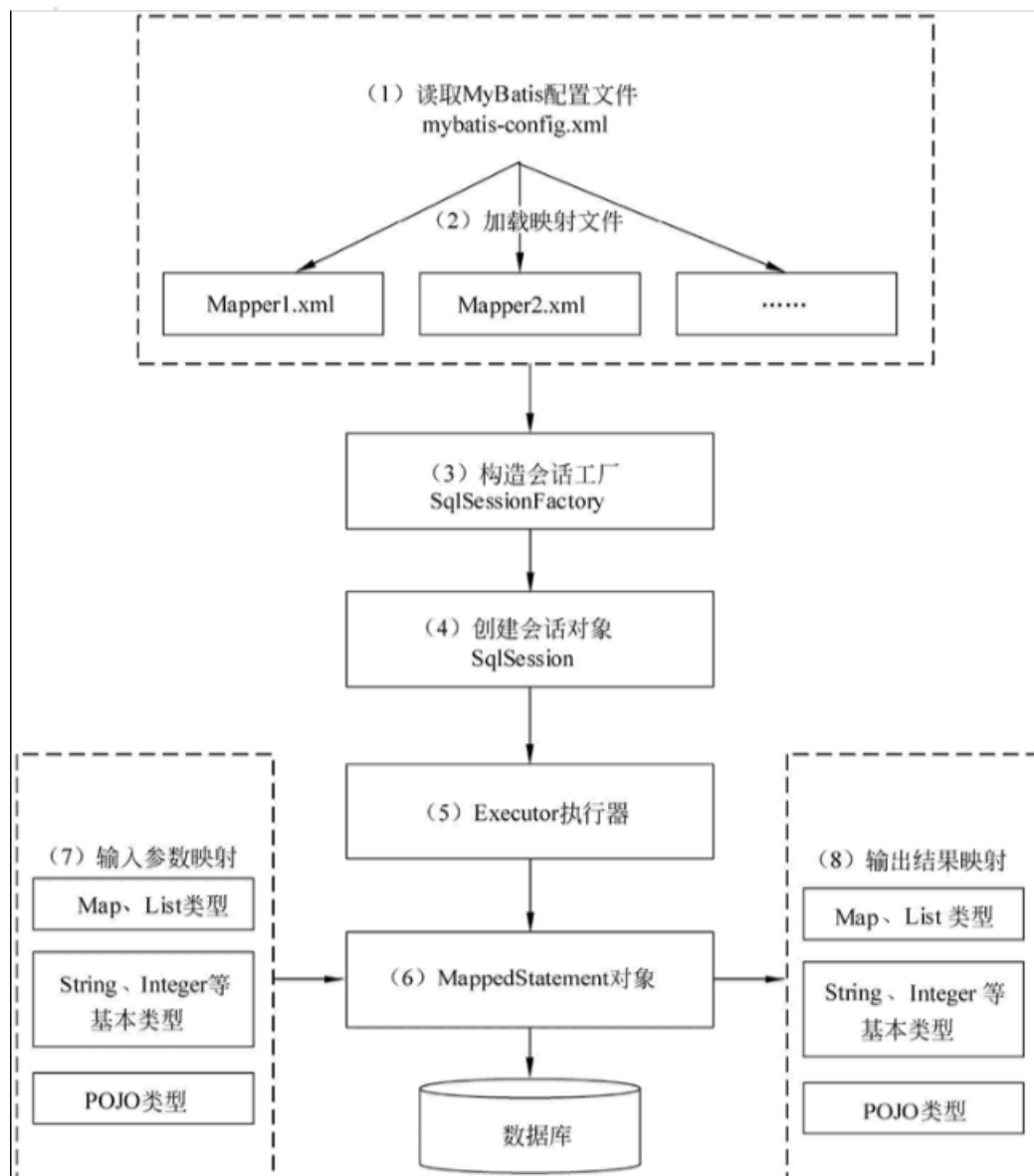
```

Run: MybatisTest x
D:\software\dev\jdk\bin\java.exe -javaagent:D:\software\d
Dept(deptno=30, dname=策划部, loc=杭州)

```

3.2、mybaits框架执行流程

执行流程图:



mybatis框架执行流程文字描述:

- 1、读取 MyBatis 配置文件：mybatis-config.xml为 MyBatis 的全局配置文件，配置了 MyBatis 的运行环境等信息，例如数据库连接信息。
- 2、加载映射文件。映射文件即 SQL 映射文件，该文件中配置了操作数据库的 SQL 语句，需要在 MyBatis 配置文件 mybatis-config.xml 中加载。mybatis-config.xml 文件可以加载多个映射文件，每个文件对应数据库中的一张表。
- 3、构造会话工厂：通过 MyBatis 的环境等配置信息构建会话工厂 SqlSessionFactory。
- 4、创建会话对象：由会话工厂创建 SqlSession 对象，该对象中包含了执行 SQL 语句的所有方法。
- 5、Executor 执行器：MyBatis 底层定义了一个 Executor 接口来操作数据库，它将根据 SqlSession 传递的参数动态地生成需要执行的 SQL 语句，同时负责查询缓存的维护。
- 6、MappedStatement 对象：在 Executor 接口的执行方法中有一个 MappedStatement 类型的参数，该参数是对映射信息的封装，用于存储要映射的 SQL 语句的 id、参数等信息。
- 7、输入参数映射：输入参数类型可以是 Map、List 等集合类型，也可以是基本数据类型和 POJO 类型。输入参数映射过程类似于 JDBC 对 preparedStatement 对象设置参数的过程。
- 8、输出结果映射：输出结果类型可以是 Map、List 等集合类型，也可以是基本数据类型和 POJO 类型。输出结果映射过程类似于 JDBC 对结果集的解析过程。

3.3、基于mapper动态代理开发

基于mapper开发的规范:

- 1、接口和mapper映射文件要在同一个目录下
- 2、接口的名称要和配置文件的名称相同(后缀名不同)
- 3、mapper映射文件namespace值必须是接口的全路径
- 4、mapper映射文件中的唯一标识id要和接口中的方法名相同
- 5、mapper映射文件中每个sql的输出映射类型要和Mapper接口中的方法返回值类型相同
- 6、mapper映射文件中每个sql的输入参数类型要和mapper接口中的参数类型相同

3.3.1、创建DeptMapper接口

```
1  /**
2   * 部门mapper接口
3   */
4  public interface DeptMapper {
5      /**根据id查询部门信息*/
6      Dept findById(Integer deptno);
7      /**根据查询所有部门数据*/
8      List<Dept> findAll();
9      /**新增部门信息*/
10     void insetDept(Dept dept);
11     /**根据id修改指定部门名称和地址*/
```

```

12 void updateDept(Dept dept);
13 /**根据id删除指定部门信息*/
14 void deleteDept(Integer deptno);
15 }

```

3.3.2、创建映射文件

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="org.wn.dao.DeptMapper">
6     <!-- 根据id查询部门信息-->
7     <select id="findById" parameterType="int" resultType="org.wn.pojo.Dept">
8         select * from dept where deptno=#{deptno}
9     </select>
10    <!-- 根据查询所有部门数据-->
11    <select id="findAll" resultType="org.wn.pojo.Dept">
12        select * from dept
13    </select>
14    <!-- 新增部门信息-->
15    <insert id="insetDept" parameterType="org.wn.pojo.Dept">
16        insert into dept values("#{deptno},#{dname},#{loc})
17    </insert>
18    <!-- 根据id修改指定部门名称和地址-->
19    <update id="updateDept" parameterType="org.wn.pojo.Dept">
20        update dept set dname=#{dname},loc=#{loc} where deptno=#{deptno}
21    </update>
22    <!-- 根据id删除指定部门信息-->
23    <delete id="deleteDept" parameterType="int">
24        delete from dept where deptno=#{deptno}
25    </delete>
26 </mapper>

```

3.3.3、修改mybatis核心配置文件

```

17 <mappers>
18     <!-- <mapper resource="org/wn/dao/DeptDao.xml"/>-->
19     <!-- 扫描持久层所在的包-->
20     <package name="org.wn.dao"/>
21 </mappers>
22 </configuration>

```

3.3.4、创建测试类

```

1 public class MybatisMapperTest {
2     public static void main(String[] args) throws IOException {
3         //1. 加载mybatis核心配置文件，创建一个SqlSessionFactory工厂类对象

```

```

4      InputStream inputStream = Resources.getResourceAsStream("mybatis-
config.xml");
5      SqlSessionFactory sqlSessionFactory = new
SqlSessionFactoryBuilder().build(inputStream);
6      //2.通过sqlSessionFactory创建sqlSession会话对象
7      SqlSession sqlSession = sqlSessionFactory.openSession();
8      //3.通过sqlSession对象获取DeptMapper的代理对象
9      DeptMapper deptMapper = sqlSession.getMapper(DeptMapper.class);
10
11     /**根据id查询部门信息*/
12     Dept dept = deptMapper.findById(30);
13     System.out.println(dept);
14     /**根据查询所有部门数据*/
15     List<Dept> list = deptMapper.findAll();
16     list.forEach(System.out::println); //遍历集合
17     /**新增部门信息*/
18     //创建Dept对象
19     Dept insertDept = new Dept(70, "教学部", "成都市");
20     deptMapper.insetDept(insertDept);
21     sqlSession.commit(); //提交事务,mybatis实现增、删、改时必须提交事务才能生效
22     /**根据id修改指定部门名称和地址*/
23     //查询指定id部门信息
24     Dept updateDept = deptMapper.findById(70);
25     //修改部门名称和地址
26     dept.setDname("国安部");
27     dept.setLoc("北京市");
28     //修改
29     deptMapper.updateDept(updateDept);
30     sqlSession.commit();
31
32     /**根据id删除指定部门信息*/
33     deptMapper.deleteDept(60);
34     sqlSession.commit();
35 }
36 }

```

3.4、Mybatis基于注解开发

在Mybatis中如果使用注解式开发，那么注解需要添加在Mapper接口中的抽象方法上，在注解中给定需要执行的SQL语句即可，这样就可

以不需要映射配置文件。Mybatis支持纯注解方式，支持纯映射配置文件方式，也支持注解和映射配置文件混合形式。当只有接口没有映

射配置文件在Mybatis-3-config.xml中对于引入映射可以通过加载指定接口类。也可以使用指定加载包。

3.4.1、使用注解完成CRUD操作

1、UserMapper接口

```
@Insert("insert into user(uid,username,password,fullname,sex,age,birthday)"
      + " values(#{uid},#{username},#{password},#{fullname},#{sex},#{age},#{birthday})")
void insert(User user);
@Update("update user set updatetime=#{updatetime}, createtime=#{createtime}, sex=#{sex} where uid=#{uid}")
void update(User user);
@Delete("delete from user where uid=#{uid}")
void delete(String uid);
@Select("select * from user where uid=#{uid}")
@Select("select uid,username,password,fullname fname,sex,age,birthday from user where uid=#{uid}")
@Results(id = "userResultId",value = {
    @Result(column = "uid",property = "uid"),
    @Result(column = "username",property = "username"),
    @Result(column = "password",property = "password"),
    @Result(column = "fname",property = "fullname"),
    @Result(column = "sex",property = "sex"),
    @Result(column = "age",property = "age"),
    @Result(column = "birthday",property = "birthday")
})
User findById(String uid);
@Select("select * from user")
List<User> findAll();
```

2、测试类

```
@Test
public void findById() {
    SqlSession session = factory.openSession();
    UserMapper userMapper = session.getMapper(UserMapper.class);
    User user = userMapper.findById("100049");
    System.out.println(user);
}
```

3.4.2、注解开发中的映射处理

在Mybatis中注解开发中，对于结果集的映射处理也是比较繁琐的，所以如果有结果集映射操作，建议使用映射配置文件方式实现

1、映射注解介绍

1.1、@Results注解

代替的标签是 <ResultMap>

@Results中的属性介绍:

id:表示唯一标识，相当于ResultMap的唯一标识

value:通过@Result注解配置映射关系

1.2、@Result注解

代替了<id>和<Result>标签

@Result中的属性介绍:

id:是否是主键字段

column:数据库列名

property:需要装配的属性名

one:需要使用@One注解(@Result(one=@One))

many:需要使用@Many注解(@Result(many=@Many))

1.3、@ResultMap注解

可通过该注解复用其它方法上的映射配置, value:指定其它@Results的id值

2、使用案例:

2.1、编辑UserMapper接口

```
@Select("select uid,username,password,fullname fname,sex,age,birthday from user where uid=#{uid}")
@Results(id = "userResultId",value = {
    @Result(column = "uid",property = "uid"),
    @Result(column = "username",property = "username"),
    @Result(column = "password",property = "password"),
    @Result(column = "fname",property = "fullname"),
    @Result(column = "sex",property = "sex"),
    @Result(column = "age",property = "age"),
    @Result(column = "birthday",property = "birthday")
})
User findById(String uid);
@Select("select uid,username,password,fullname fname,sex,age,birthday from user")
@ResultMap("userResultId") // 复用其它@Results映射关系处理
List<User> findAll();
```

2.2、测试类

```
@Test
public void findAll() {
    SqlSession session = factory.openSession();
    UserMapper userMapper = session.getMapper(UserMapper.class);
    List<User> users = userMapper.findAll();
    System.out.println(users);
}
```

3.4.3、注解开发中使用动态SQL

在Mybatis中的注解开发中,对于动态SQL的处理是比较繁琐的,所以如果有动态SQL的操作,建议使用映射配置文件方式实现。

1、使用脚本SQL实现

在 `<script>` 标签中通过动态SQL的标签完成SQL的拼接。

1.1、在UserMapper接口定义方法

```
@Select("<script>select * from user <where>"
+ "<if test=\"fullname!=null and fullname!='\">and fullname=#{fullname}</if>"
+ "<if test=\"sex!=null and sex!='\">and sex=#{sex}</if>"
+ "</where></script>")
List<User> selectByUsers(User user);
```

1.2、测试类

```
@Test
public void selectByUsers() {
    SqlSession session = factory.openSession();
    UserMapper userMapper = session.getMapper(UserMapper.class);
    //实体类进行输入参数映射
    User user = new User();
    user.setSex("女");
    user.setFullname("上官婉儿");
    List<User> ulist = userMapper.selectByUsers(user);
    ulist.forEach(System.out::println);
}
```

2、在方法中构建SQL

在Mybatis的注解中包含了@SelectProvider、@UpdateProvider、@DeleteProvider、@InsertProvider,这些注解统称为SqlProvider,它

们分别对应这查询、修改、删除、插入操作。当使用这些注解时将不在注解中注解直接编写SQL, 而是调用某个类的特定方法类生成

SQL。

2.1、修改UserMapper接口

```
// @Select("<script>select * from user <where>"
//      + "<if test=\"fullname!=null and fullname!='\">and fullname=#{fullname}</if>"
//      + "<if test=\"sex!=null and sex!='\">and sex=#{sex}</if>"
//      + "</where></script>")
// @SelectProvider(type =com.mapper.UserMapperProvider.class,method = "queryUserProviderSQL")
List<User> selectByUsers(User user);|
```

2.2、定义UserMapperProvider类

```
public String queryUserProviderSQL(User user) {
    //定义可变字符串对象
    StringBuffer sb = new StringBuffer("select * from user where 1=1 ");
    //判断fullname属性和sex属性是否有值
    if(user.getFullname() != null && user.getFullname().trim() != "") {
        sb.append(" and fullname=#{fullname}");
    }
    if(user.getSex() != null && user.getSex().trim() != "") {
        sb.append(" and sex=#{sex}");
    }
    System.out.println(sb.toString());
    return sb.toString();
}
```

2.3、测试类

```
@Test
public void selectByUsers() {
    SqlSession session = factory.openSession();
    UserMapper userMapper = session.getMapper(UserMapper.class);
    //实体类进行输入参数映射
    User user = new User();
    user.setSex("女");
    user.setFullname("上官婉儿");
    List<User> uList = userMapper.selectByUsers(user);
    uList.forEach(System.out::println);
}
```

四、Mybatis核心配置文件

4.1、标签properties

4.1.1、创建properties文件

```
1 jdbc.driver=com.mysql.jdbc.Driver
2 jdbc.url=jdbc:mysql://localhost:3306/mybatis_database
3 jdbc.username=root
4 jdbc.password=19901030
```

4.1.2、核心配置文件中引入properties文件

```
<configuration>

    <!-- 配置properties元素引用外部的properties文件 -->
    <properties resource="db.properties"></properties>

    <!-- 配置数据库数据源 -->
    <environments default="development">
        <environment id="development">
            <transactionManager type="JDBC"/>
            <dataSource type="POOLED">
                <property name="driver" value="${jdbc.driver}"/>
                <property name="url" value="${jdbc.url}"/>
                <property name="username" value="${jdbc.username}"/>
                <property name="password" value="${jdbc.password}"/>
            </dataSource>
        </environment>
    </environments>
</configuration>
```

4.2、setting标签(全局设置)

这是MyBatis 中极为重要的调整设置，它们会改变 MyBatis 的运行时行为(影响全局)。下表描述了设置中各项的意图、默认值等：

设置名	描述	有效值	默认值
cacheEnabled	全局地开启或关闭配置文件中的所有映射器已经配置的任何缓存。	true false	true
lazyLoadingEnabled	延迟加载的全局开关。当开启时，所有关联对象都会延迟加载。特定关联关系中可通过设置 fetchType 属性来覆盖该项的开关状态。	true false	false
aggressiveLazyLoading	当开启时，任何方法的调用都会加载该对象的所有属性。否则，每个属性会按需加载（参考 lazyLoadTriggerMethods）。	true false	false （在 3.4.1 及之前的版本默认值为 true）
defaultStatementTimeout	设置超时时间，它决定驱动等待数据库响应的秒数。	任意正整数	未设置 (null)
safeRowBoundsEnabled	允许在嵌套语句中使用分页（RowBounds）。如果允许使用则设置为 false。	true false	false
safeResultHandlerEnabled	允许在嵌套语句中使用分页（ResultHandler）。如果允许使用则设置为 false。	true false	true
lazyLoadTriggerMethods	指定哪个对象的方法触发一次延迟加载。	用逗号分隔的方法列表。	equals,clone,hashCode,toString
defaultScriptingLanguage	指定动态 SQL 生成的默认语言。	一个类型别名或完全限定类名。	org.apache.ibatis.scripting.xmltags.XMLLanguageDriver
defaultFetchSize	为驱动的结果集获取数量（fetchSize）设置一个提示值。此参数只可以在查询设置中被覆盖。	任意正整数	未设置 (null)
multipleResultSetsEnabled	是否允许单一语句返回多结果集（需要驱动支持）。	true false	true

4.3、typeAliases(类型别名)

下面是一些为常见的 Java 类型内建的类型别名。它们都是不区分大小写的，注意，为了应对原始类型的命名重复，采取了特殊的命名风格。

<code>_int</code>	<code>int</code>
<code>_integer</code>	<code>int</code>
<code>_double</code>	<code>double</code>
<code>_float</code>	<code>float</code>
<code>_boolean</code>	<code>boolean</code>
<code>string</code>	<code>String</code>
<code>byte</code>	<code>Byte</code>
<code>long</code>	<code>Long</code>
<code>short</code>	<code>Short</code>
<code>int</code>	<code>Integer</code>
<code>integer</code>	<code>Integer</code>
<code>double</code>	<code>Double</code>
<code>float</code>	<code>Float</code>
<code>boolean</code>	<code>Boolean</code>
<code>date</code>	<code>Date</code>
<code>decimal</code>	<code>BigDecimal</code>
<code>bigdecimal</code>	<code>BigDecimal</code>
<code>object</code>	<code>Object</code>
<code>map</code>	<code>Map</code>
<code>hashmap</code>	<code>HashMap</code>
<code>list</code>	<code>List</code>
<code>arraylist</code>	<code>ArrayList</code>
<code>collection</code>	<code>Collection</code>
<code>iterator</code>	<code>Iterator</code>

类型别名在配置文件中可以这样使用：

4.4、mappers标签配置

Resource和class属性配置如下：

```
<mappers>
  <!-- <mapper resource="com/xmkj/entity/userMapper.xml"/> -->
  <mapper resource="com/xmkj/dao/userDao.xml"/>
  <!-- <mapper resource="com/xmkj/mapper/userMapper.xml"/> -->
  <mapper class="com.xmkj.mapper.UserMapper"/>
</mappers>
```

五、Mybatis之sql映射文件

5.1、输入映射

在mybatis中参数传递三种方式。如下:

5.1.1、顺序传参法

注解案例:

```
//根据fullname和sex参数进行条件查询
@Select("select * from user where fullname=#{param1} and sex=#{param2}")
List<User> selectByUsers(String fullname,String sex);
```

配置文件案例:

```
<select id="selectByUsers" parameterType="String" resultType="User">
  select * from user where fullname=#{param1} and sex=#{param2}
</select>
```

5.1.2、POJO传参法

注解开发案例:

```
@Select("select * from user where fullname=#{fullname} and sex=#{sex}")
List<User> selectByUsers(User user);
```

配置文件案例:

```
<select id="selectByUsers" parameterType="User" resultType="User">
  select * from user where fullname=#{fullname} and sex=#{sex}
</select>
```

5.1.3、Map传参法

注解开发案例:

```
@Select("select * from user where fullname=#{fullname} and sex=#{sex}")
List<User> selectByUsers(Map<String,String> map);
```

注意:Map传参, 根据key获取value方式取值

5.1.4、@Param注解传参

在什么情况使用@Param注解呢？

- 1.当方法有多个参数时。（最常用）
- 2.想给方法的参数取别名。（可以将复杂的参数名称换个简单的）
- 3.当xml文件的sql语句含有占位符（占位符稍后会讲解）
- 4.如果在动态 SQL 中使用了参数作为变量，那么也需要 @Param 注解，即使你只有一个参数。

案例：

```
/**根据部门名称和地址查询部门信息*/
@Select(" select * from dept where dname=#{dname} and loc=#{loc}") 1 usage
List<Dept> selectByNameAndLoc(@Param("dname") String dname,@Param("loc") String loc);
```

5.2、Mybatis输出映射

5.2.1、在mapper配置文件中配置如下：

```
<!-- 创建一个resultMap -->
<resultMap type="User" id="userResultMap">
    <!-- 配置主键 -->
    <id column="_uid" property="uid"></id>
    <!-- 配置其它属性 -->
    <result column="_username" property="username"/>
</resultMap>

<!-- 根据id号查询数据 -->
<select id="findUserById" parameterType="int" resultMap="userResultMap">
    select uid _uid,username _username,sex,address from t_user where uid=#{uid}
</select>

<!-- 根据用户的名称进行模糊查询 -->
```

5.2.2、编写测试类

```
@Ignore
public void findUserById(){
    //1.获取回话对象
    SqlSession sqlSession = sqlSessionFactory.openSession();
    //2.获取mapper的代理对象
    UserMapper mapper = sqlSession.getMapper(UserMapper.class);
    //3.执行查询操作
    User user = mapper.findUserById(6);
    System.out.println(user);
}
```

5.3、Mybatis动态SQL

在jdbc中我们经常会遇到字符串拼接sql语句，然后把参数传入再查询等等。而动态sql的出现就是为了解决这个拼接的问题。

5.3.1、mybatis常见的动态标签

1 | <if>、<choose>、<foreach>、<where>、<trim>等

5.3.2、if判断(重点掌握)

```
<!-- 定义sql片段 -->
<sql id="query_where_string">
    <where>
        <if test="name !=null and name !=''">
            username LIKE '%${name}%'
        </if>
        <if test="orderby !=null and orderby !=''">
            ORDER BY username ${orderby}
        </if>
    </where>
</sql>
<!-- 测试一个多条件查询 -->
<select id="findUserList" resultType="User" parameterType="User">
    select * from t_user
    <!-- 引入sql片段 -->
    <include refid="query_where_string"></include>
</select>
```

5.3.3、sql片段使用(掌握)

```
<!-- 自定义SQL片段 -->
<sql id="queryUserId">
    <where>
        <if test="fullname != null and fullname != ''">
            and fullname = #{fullname}
        </if>
        <if test="sex != null and sex != ''">
            and sex = #{sex}
        </if>
        <if test="password != null and password != ''">
            and password = #{password}
        </if>
    </where>
</sql>
```

5.3.4、forEach(了解)